Saudi HPC/AI 2026

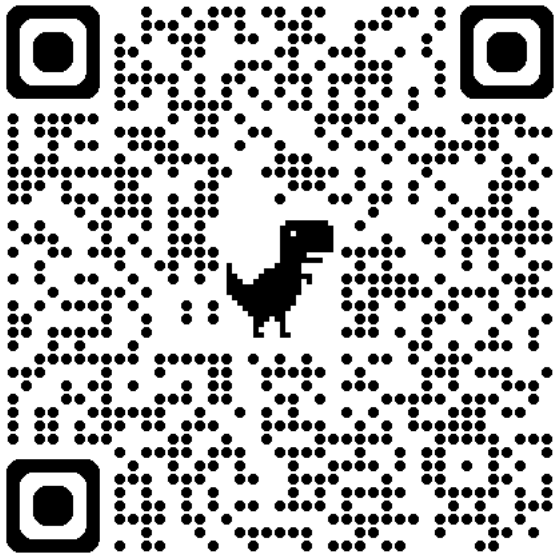
Hands-on MLOps with Kubeflow

Taking AI/ML models from Experiment to Production

Mohsin Ahmed Shaikh

AI/ML Support Team Lead @ KAUST Supercomputing Lab

Get your access



1. Scan the QR code
2. Fill in the form to request your access credentials
3. Check your email for the notification with your cluster and login
4. Maximum accounts – 80 (🤲 touchwood)

Form link: https://forms.gle/afK1AiFJkm2ju3SX8

Agenda

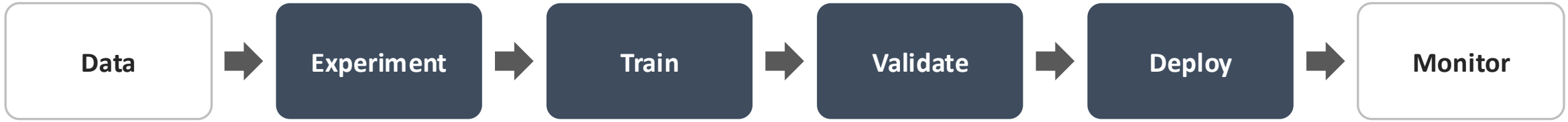
Time	Session	Min
1:30 PM	Introduction	10
1:40 PM	The story and the tools: the sick leaf, MLOps, Kubeflow	15
1:55 PM	The terrain: your cluster, GPUs and the queue	10
2:05 PM	Exercise 1, Notebooks: workspace and baseline	30
2:35 PM	Exercise 2, Katib: tune the settings	30
3:05 PM	Break	10
3:15 PM	Exercise 3, Trainer: scale up the winner	30
3:45 PM	Exercise 4, Pipelines: a gate before production	30
4:15 PM	Exercise 5, KServe: from a file to a service	20
4:35 PM	Closing and feedback	5

Things to know before you start!

- Email notification includes:
  - Cluster URL with Kubeflow login page [aws-#.mlops101.io]
  - Username and Password (case-sensitive)
    - Please don’t share
  - Monitoring: [aws-#.mlops101.io]/grafana
    - Same credentials
- Each notebook gets 1 T4 (16GB) GPU
  - Max 2 GPU quota for batch jobs (20 participants share 8 GPUs – be patient)
- Code: https://github.com/kaust-rccl/saudihpc-tutorials.git

MLOps: from one good run to a service

MLOps applies DevOps habits to machine learning: every step repeatable, automated and observable.



Today we walk from Experiment to Deploy. Data pipelines and monitoring are the next steps.

What goes wrong without it

It worked on my laptop

Settings and data live in one person's notebook and nobody can repeat the run.

Which model is live?

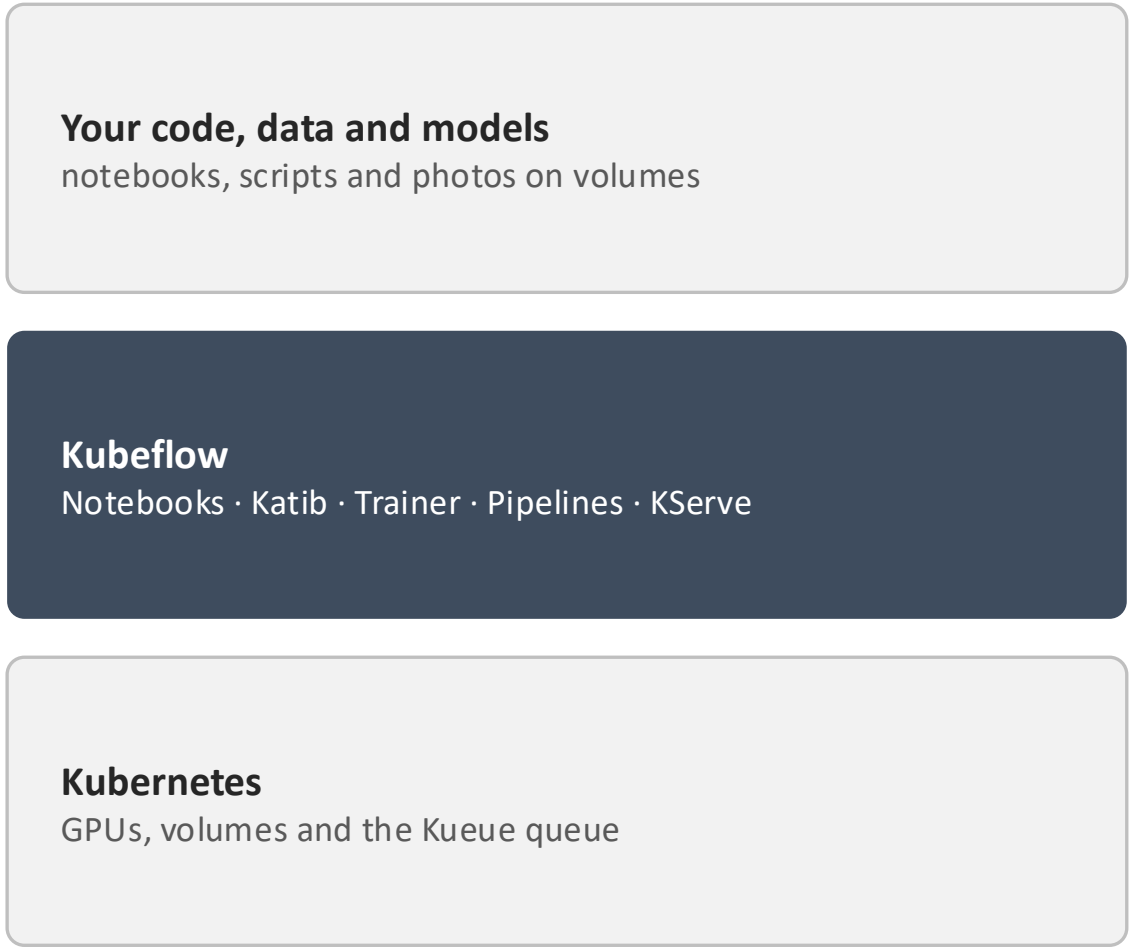
Nobody can say what was trained, on which data, or how well it scored.

Shipped by hand

Steps get skipped, and a weak model can reach the people who rely on it.

Kubeflow: the ML toolkit for Kubernetes

- Open-source platform for the machine-learning lifecycle
- Built on Kubernetes, so it runs on any cluster with GPUs
- One tool for each step: notebooks, tuning, training, pipelines, serving
- Each tool is a Kubernetes resource: click it in the dashboard or write it as YAML



One Kubeflow tool for each lifecycle step

Component	What it does	Exercise
Notebooks	Your GPU workspace in the browser, for exploring and prototyping	1
Katib	Tries settings automatically and reports the best ones	2
Trainer	Runs training as a background batch job, on one or many GPUs	3
Pipelines	Chains steps into a repeatable workflow, with a gate before release	4
KServe	Turns a saved model into a live web service	5

Underneath all of them, Kueue queues batch work and shares the GPUs fairly between participants.

What this means in practice

1

Declare, do not babysit

You describe the job: settings, GPUs, volumes. Kubernetes starts it, restarts it if it fails, and keeps the logs.

2

Share the GPUs

Your notebook has its own T4. Trials, training runs and pipeline steps wait in a shared queue and start when a GPU is free.

3

Keep work on volumes

Pods come and go. Code, data and models live on volumes, so every step finds them again.

Your training setup

1. Reach your cluster

Four clusters run the tutorial. The notification email names yours.

```
aws.mlops101.io
aws-w.mlops101.io
aws-sg.mlops101.io
aws-in.mlops101.io
```

Check your email now and note which one is yours.

2. Your workspace

- A notebook server with one T4 GPU
- Volume home at /home/jovyan: your files and the code
- Volume data at /data: the photos (already loaded) and the saved models
- Batch work in Exercises 2 to 5 runs in the shared GPU queue

3. Where the code is

GitHub: kaust-rccl/saudihpc-tutorials, branch dev, folder kubeflow-tutorial

```
cd /home/jovyan && git clone -b dev https://github.com/kaust-rccl/saudihpc-tutorials.git
Scripts run from /home/jovyan/saudihpc-tutorials/kubeflow-tutorial/exercise<N>/
```

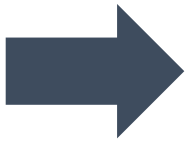
The story of the sick leaf

A farmer photographs a leaf. The app must say whether it is healthy or sick, and which condition it is.

Lab photos

PlantVillage

- Leaves on a plain background
- 38 conditions, healthy and sick
- We train and test the model here



Field photos

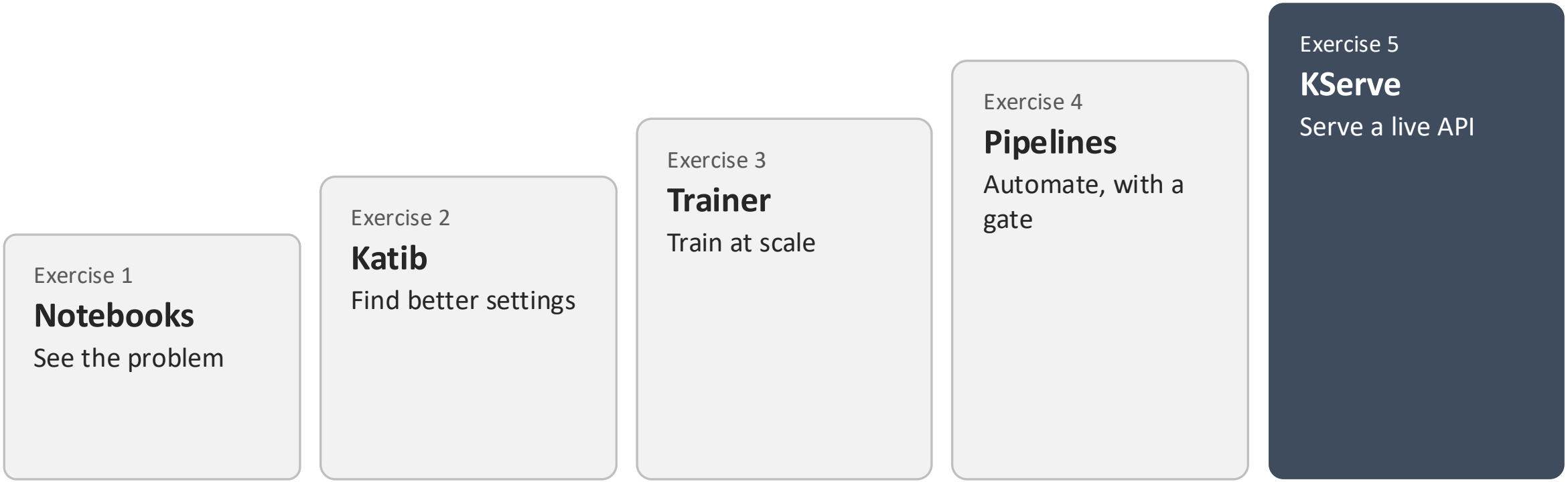
RoCoLe coffee leaves

- Coffee leaves, photographed on the plant
- The model never trains on these
- They test one question: healthy or sick?

Being 97% right in the lab says little about the field. We will measure the gap, improve the model, and ship only what holds up.

Our ambition for the next three hours

Turn one notebook experiment into a service the app can call, with a gate that keeps weak models out.



End result: send a leaf photo, get back how sick it looks, which condition, or "refer to an extension officer" when the model is unsure.

Exercise 1: Notebooks, see the problem

Kubeflow component: Kubeflow Notebooks

What we do

- Open a GPU notebook and check the workspace and the data volume
- Fine-tune ConvNeXt-Tiny on lab photos of leaves: the baseline
- Test it on coffee-leaf photos from the field, then try an augmentation fix

Why we do it

- Before we automate anything, we must know whether the model works where it will be used
- High accuracy on lab photos can hide a weak model

Intended outcome

- A baseline with about 97% accuracy on lab photos
- A field test that shows what the lab number hides
- A first improvement from data augmentation

Lessons learnt

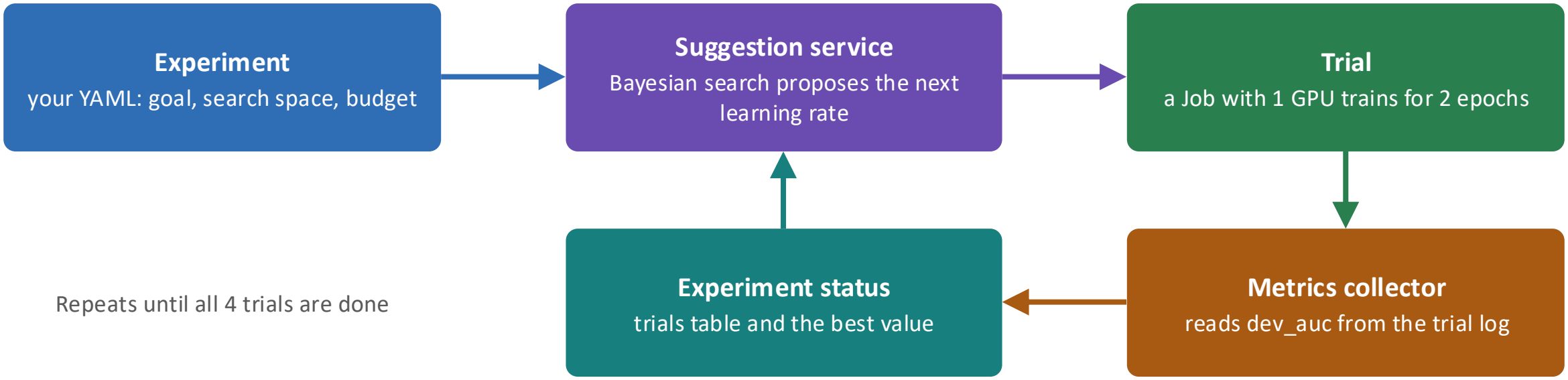
- Lab accuracy is not field accuracy
- No leaf or plant may sit on both sides of a split, or the scores lie
- One run is not proof: results move with the random seed

Exercise 1: Notebooks, see the problem

- Follow me for:
  - Create Volumes
    - home
    - data
    - dataset – readonly mount
  - Starting a notebook
    - Select the right image
    - Select compute resource (CPU, Mem, GPU)
    - Mount volumes:
      - home -> /home
      - data -> /data
      - dataset -> /datasets
  - Create notebook

Katib: automated hyperparameter tuning

You state the goal, search space and budget; Katib runs the trials.



You provide

The objective, the parameter range and the trial template: what one trial runs.

Katib runs

Trials as Kubernetes Jobs in the GPU queue, one after another.

You get

A trials table in the Katib UI and the best learning rate.

Katib in YAML: the Experiment

```yaml
apiVersion: kubeflow.org/v1beta1
kind: Experiment
spec:
  objective:                        # what "best" means
    type: maximize
    objectiveMetricName: dev_auc    # printed by every trial
  algorithm:
    algorithmName: bayesianoptimization
  parameters:                       # what Katib may change
  - name: lr
    parameterType: double
    feasibleSpace: {min: "0.00003", max: "0.001"}
  maxTrialCount: 4                  # the budget
  parallelTrialCount: 1             # one trial at a time
  trialTemplate:                    # what one trial runs
    trialSpec:                      # a Kubernetes Job, 1 GPU
      ...
      args: [--lr, "${trialParameters.learningRate}", --epochs, "2"]
```

objective
What "best" means: maximise dev_auc, which every trial prints.

parameters
What Katib may change: the learning rate, 3e-5 to 1e-3.

maxTrialCount
The budget: 4 trials, one at a time.

trialTemplate
What one trial runs: a Job with 1 GPU, queued by Kueue.

Exercise 2: Katib, tune the settings

Kubeflow component: Katib

What we do

- Paste an Experiment YAML into the Katib UI: tune the learning rate for the best field score
- Katib runs 4 short trials (2 epochs) and picks each next value by Bayesian search
- Read the trials table and pick the winner

Why we do it

- Tuning by hand is slow and hard to repeat
- Each trial is a small job: the cluster runs them for you, one at a time, in the shared queue

Intended outcome

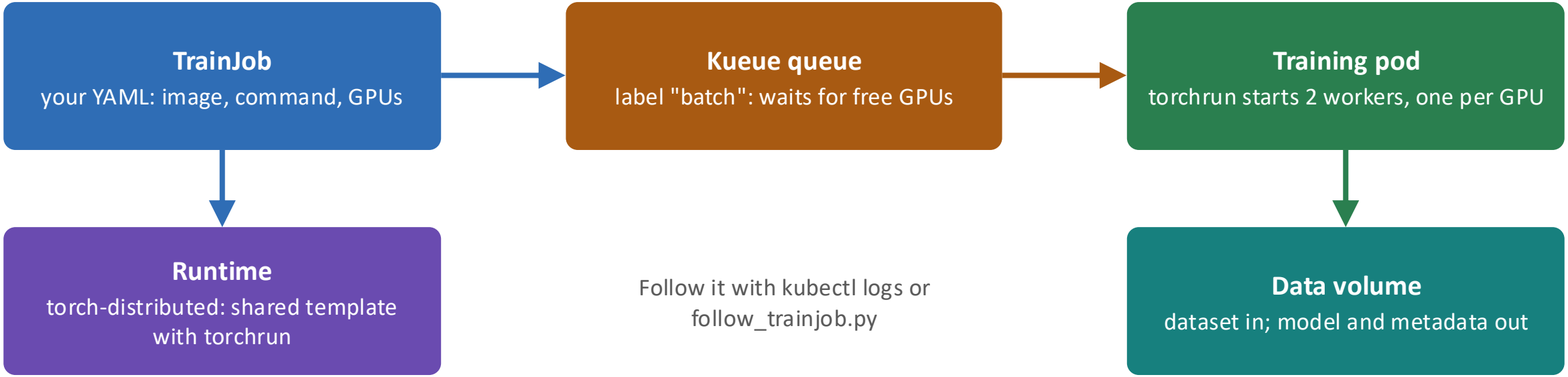
- A table of trials and the best learning rate
- The setting we carry into Exercise 3: learning rate 0.0003

Lessons learnt

- State the setting, the goal and the budget before you start
- Differences of a few hundredths are noise: do not over-read them
- Short trials keep GPU time low

Trainer: training as a batch job

You submit a TrainJob; Trainer starts the workers on one GPU or several.



You provide

A TrainJob YAML: image, command, number of GPUs and the training settings.

Trainer runs

The workers as a batch job in the queue; PyTorch DDP splits each batch over the GPUs.

You get

Live logs, and a saved model with its metadata.json.

Trainer in YAML: the TrainJob

```yaml
apiVersion: trainer.kubeflow.org/v1alpha1
kind: TrainJob
metadata:
  name: cropguard-2gpu
  labels: {kueue.x-k8s.io/queue-name: batch}   # wait in the GPU queue
spec:
  runtimeRef:
    name: torch-distributed        # shared, ready-made runtime
  trainer:
    numNodes: 1                    # one pod, so one node
    numProcPerNode: 2              # two workers, one per GPU
    resourcesPerNode:
      limits: {nvidia.com/gpu: "2"}
    command: [torchrun, .../exercise3/train_ddp.py]
    args: [--lr, "0.0003", --epochs, "2", --name, t4-2gpu]
  runtimePatches:                  # adds the volumes to the pods
  - ...
```

runtimeRef
The shared runtime that supplies torchrun and the rank setup.

numProcPerNode
Two workers in one pod, so both GPUs are on one node.

queue label
The job waits until Kueue has free GPUs.

runtimePatches
Adds the data and home volumes to the training pod.

Exercise 3: Trainer, scale up the winner

Kubeflow component: Kubeflow Trainer

What we do

- Submit a TrainJob from a YAML file: 1 GPU first, then 2 GPUs on one node
- Follow the live logs of each worker
- The job saves the model and a metadata file on your data volume

Why we do it

- Real training should not tie up your notebook: it runs in the background, in the queue
- Trainer starts the workers; the same script runs on 1 or 2 GPUs

Intended outcome

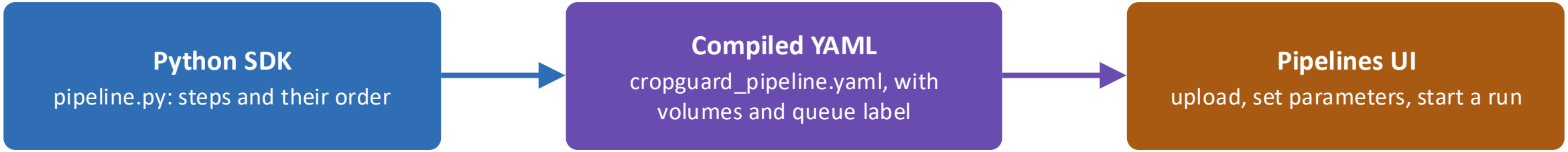
- A trained model in /data/models/cropguard/<run>/
- Your own numbers: photos per second on 1 and on 2 GPUs

Lessons learnt

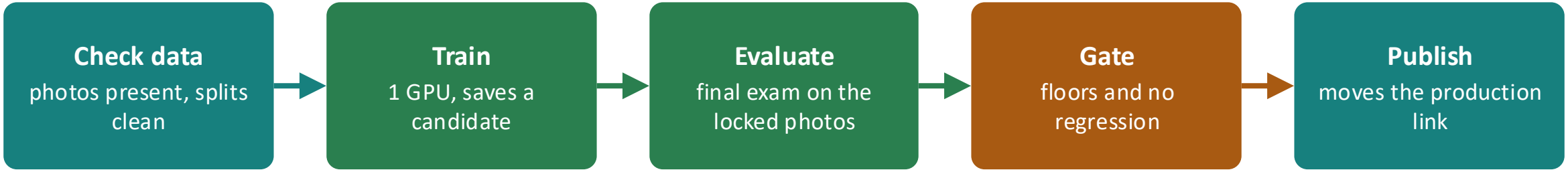
- Batch work: submit, queue, follow the logs, collect the result
- Scaling buys speed and workflow, not a better model
- Both GPUs sit on one node, so they can talk fast

Pipelines: a workflow you can rerun

Write the steps once in Python, compile, upload, run: every run is saved.



A run executes five steps, each in its own pod



GPU step, waits in the queue

CPU step

decision: publish runs only if the gate passed

Pipelines in Python: the KFP SDK

```python
from kfp import compiler, dsl

@dsl.container_component
def train(data_dir: str, lr: float, epochs: int):
    return dsl.ContainerSpec(image=image,
        command=['python', train_py],
        args=['--data', data_dir, '--lr', lr, '--epochs', epochs])

@dsl.pipeline(name='cropguard-train-evaluate-gate-publish')
def cropguard(lr: float = 0.0003, epochs: int = 2, ...):
    t1 = check_data(...)
    t2 = train(lr=lr, epochs=epochs, ...).after(t1)
    t2.set_accelerator_type('nvidia.com/gpu').set_accelerator_limit(1)
    t3 = evaluate(...).after(t2)
    t4 = gate(...).after(t3)
    with dsl.If(t4.outputs['decision'] == 'passed'):
        publish(...)

compiler.Compiler().compile(cropguard, 'cropguard_pipeline.yaml')
```

```
# 1. compile to YAML
python pipeline.py --out cropguard_pipeline.yaml
# 2. submit the run (needs the token) or upload the YAML in the UI
python run_pipeline.py
```

container_component
One step is one container that runs our script.

pipeline
The function body is the graph: order and conditions.

.after(...)
Sets the order; GPU steps ask for one GPU.

dsl.If
Publish runs only when the gate says passed.

compile
Writes the YAML that you upload in the UI.

Exercise 4: Pipelines, a gate before production

Kubeflow component: Kubeflow Pipelines

What we do

- Compile the pipeline, upload it in the Pipelines UI and start a run
- Watch the five steps run: check data, train, evaluate, gate, publish
- Set an impossible bar and watch the gate block the model

Why we do it

- Manual steps get skipped or done differently each time
- Every model should face the same exam before anyone relies on it

Intended outcome

- A model reaches production only if it passes the gate
- A blocked model stays on disk and production does not change
- Every run leaves a record of its settings and verdict

Lessons learnt

- A pipeline is a recipe: each step runs in its own pod
- The locked coffee photos are scored once, in the final exam only
- The gate leaves a margin for run-to-run noise instead of blocking on luck

KServe: from a model folder to a service

- An InferenceService becomes a Deployment, a Service and health checks
- One standard request: POST /v1/models/cropguard:predict with a photo
- Our server loads the model that the production link points to
- Publish or roll back: the live service switches, with no redeploy
- RawDeployment mode: a plain Deployment, which can mount our volumes

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: cropguard
  annotations:
    serving.kserve.io/deploymentMode: RawDeployment
spec:
  predictor:
    containers:
    - name: kserve-container      # the name KServe expects
      image: <the notebook image>
      command: [python, .../exercise5/serve_cropguard.py]
      env:
      - {name: MODEL_DIR,
         value: /data/models/cropguard/production}
      ports: [{containerPort: 8080}]
      volumeMounts:               # data and home, read-only
      - {name: data, mountPath: /data, readOnly: true}
      - {name: home, mountPath: /home/jovyan, readOnly: true}
```

KServe: from a model folder to a service

Deployment modes

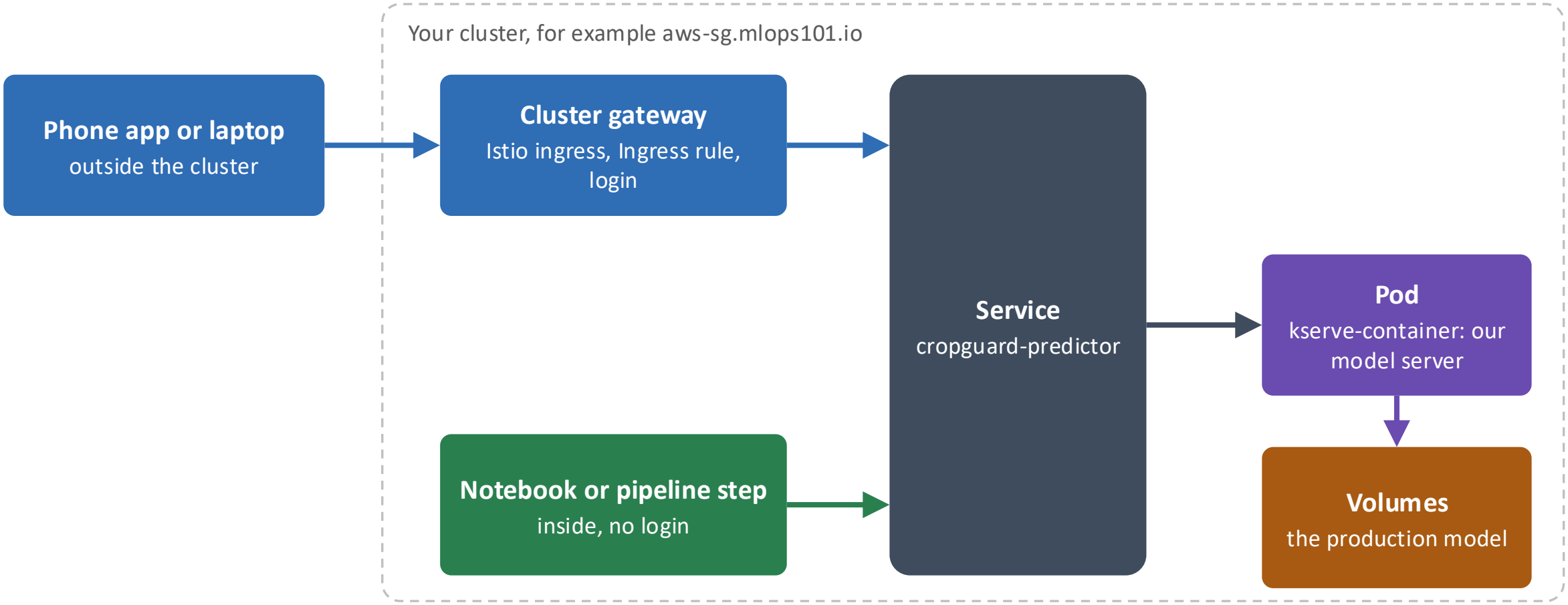
- Serverless (Knative): scales to zero, autoscales on requests
- RawDeployment: plain Deployment + Service, can mount our volumes
- ModelMesh: many small models sharing pods

Serving runtimes

- Built in: Hugging Face, Triton, TorchServe, TensorFlow Serving, scikit-learn, XGBoost, MLServer
- Custom container: ours, serve_cropguard.py
- LLMInferenceService: large language models

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: cropguard
  annotations:
    serving.kserve.io/deploymentMode: RawDeployment
spec:
  predictor:
    containers:
    - name: kserve-container      # the name KServe expects
      image: <the notebook image>
      command: [python, .../exercise5/serve_cropguard.py]
      env:
      - {name: MODEL_DIR,
         value: /data/models/cropguard/production}
      ports: [{containerPort: 8080}]
      volumeMounts:               # data and home, read-only
      - {name: data, mountPath: /data, readOnly: true}
      - {name: home, mountPath: /home/jovyan, readOnly: true}
```

KServe: how a request reaches the model



outside `https://aws-sg.mlops101.io/serving/<namespace>/cropguard/v1/models/cropguard:predict`

inside `http://cropguard-predictor.<namespace>.svc.cluster.local/v1/models/cropguard:predict`

Exercise 5: KServe, from a file to a service

Kubeflow component: KServe

What we do

- Deploy an InferenceService that serves the model in production
- Send leaf photos from a small client, inside the cluster
- Publish a new model and call again

Why we do it

- The app needs an address to send a photo to, not a file on a disk
- KServe adds health checks and one standard request format

Intended outcome

- A live API: photo in; sick score, sick or healthy, top 3 conditions
- "Refer to an extension officer" when the model is unsure

Lessons learnt

- The sick/healthy cut-off travels with the model
- "production" is a pointer: publish or roll back with no redeploy
- Know the limits: for coffee leaves only sick-or-healthy applies